ADVANCED DATABASE SYSTEMS

INDIVIDUAL PRACTICAL DATABASE PORTFOLIO

Student	Oditah Oluchukwu Michael
Student ID	301920236
Case study	Meridian Digital Bank (simulated)

1. Case study and portfolio scope

Meridian Digital Bank is a fictional Nigerian digital bank used consistently across the five portfolio activities. The system supports customer onboarding, account servicing, internal transfers, double-entry ledger posting, statements and audit review. Its users include customers, support officers, operations staff, auditors and authorised banking services. The test dataset is wholly synthetic.

2. Database design and modelling

The relational model separates branches, customers, accounts, transfers, ledger entries and audit logs. Primary/foreign keys, unique constraints and CHECK constraints protect entity integrity and valid banking states. Monetary values use integer kobo to avoid floating-point rounding. Transfers create debit and credit ledger entries of equal value, while account balance changes and audit records are maintained transactionally.

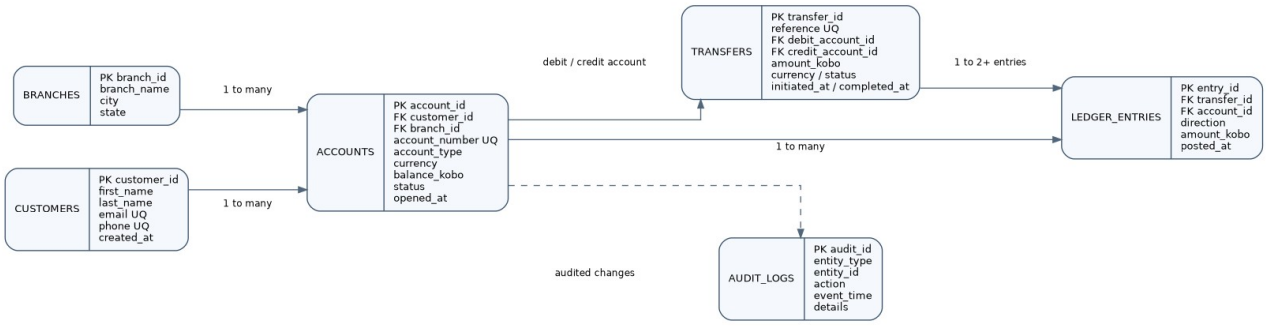


Figure 1. Meridian Digital Bank entity-relationship model.

The structure targets Third Normal Form: customer, branch and transfer facts are kept in their own tables; ledger entries are stored as atomic records rather than repeating groups. The account balance is a deliberate operational projection that must be updated in the same transaction as ledger postings and checked through reconciliation.

3. Query processing and optimisation

The principal performance test is an account statement filtered by account ID and posting-time range, ordered newest-first. The index places the equality key first and the time key second; the selected direction and amount columns are also added to support a covering-index plan where SQLite can use one.

Before composite index	After composite index
SCAN ledger_entries	SEARCH ledger_entries USING COVERING INDEX idx_ledger_account_posted (account_id=? AND posted_at>? AND posted_at<?)

In the local synthetic test, the median of 150 executions changed from 0.9349 ms to 0.0586 ms. This is a machine- and cache-dependent micro-benchmark, not a production service-level guarantee. The full plan and timing evidence are included in the repository.

4. Transactions and concurrency

The local SQLite implementation performs account validation, balance debit/credit, transfer insertion and two ledger postings in one BEGIN IMMEDIATE transaction. The test evidence records a committed transfer, a deliberate rollback that leaves both balances unchanged, rejection of an insufficient-funds transfer, and a second writer blocked while the first writer holds the write reservation. SQLite serialises writers; the separate PostgreSQL example illustrates row-level SELECT FOR UPDATE locking and a commit boundary.

- COMMIT: transfer TRF-DEMO-C77D0CE8E6FB committed; debit account reduced by 25000 kobo: 99486335 -> 99461335
- ROLLBACK: staged 1000-kobo debit/credit was rolled back; balances unchanged (99461335=99461335; 101135404=101135404)
- REJECTION: Insufficient funds; transaction rolled back

5. NoSQL and advanced data model

The MongoDB design embeds frequently accessed contact, KYC and account-summary fields in a customer profile for dashboard reads. It keeps the full transfer event history in a separate collection because that history grows and should be paginated. A unique transfer reference supports idempotency; a compound customer/time index supports recent-activity queries. The supplied mongosh script includes inserts, indexes, a read query, explain output and a multi-document transaction. The JSON sample was validated locally, but MongoDB Server/mongosh was not available for execution in the authoring environment; genuine server screenshots must be added after the student runs it.

6. Distributed and cloud database design

The proposed cloud architecture uses a managed PostgreSQL writer across multiple Availability Zones, a standby for high availability, and reader capacity for non-critical history and reporting. Read replicas can lag, so the writer remains the authority for transfer confirmation and balance reads immediately after a write. Large ledger tables can first be partitioned by posting month; horizontal sharding is deferred until a measured scale requirement justifies the complexity. Any cross-shard transfer needs an explicit distributed workflow, idempotency and reconciliation. This is a design exercise, not a live cloud deployment.

7. Technical reflection and AI-use declaration

The design uses relational constraints and transactions for financial invariants, while the document model serves flexible customer-facing profile data. Indexes improve targeted reads but consume storage and add write work. The student should replace this paragraph with a personal reflection describing the work personally executed, the challenges actually encountered, and how the results were verified.

AI-use declaration: ChatGPT (OpenAI) assisted with portfolio structuring, draft explanations and example scripts. The student must review, correct and understand all materials and update the declaration to reflect actual personal testing. The SQLite evidence was executed locally; MongoDB, PostgreSQL server and cloud provisioning were not executed in the authoring environment.

References

1. Amazon Web Services. (n.d.). Amazon RDS User Guide. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
2. MongoDB. (n.d.). Embedded data; Transactions; Sharding. https://www.mongodb.com/docs/manual/
3. PostgreSQL Global Development Group. (n.d.). Using EXPLAIN. https://www.postgresql.org/docs/current/using-explain.html
4. PostgreSQL Global Development Group. (n.d.). Explicit locking. https://www.postgresql.org/docs/17/explicit-locking.html

GitHub repository: [PASTE YOUR PRIVATE REPOSITORY URL HERE BEFORE SUBMISSION]